

I actually think we're reaching the point where these documents become developer-ready. This next one is especially important if Replit Agent is building Version 1, because it removes a lot of ambiguity about how different parts of the app communicate.

Assignment Guy

Document 6

API & Backend Specification

Version 1.0

1. Purpose

This document defines the backend services and APIs required for Assignment Guy Version 1.

The backend should expose clear, predictable endpoints that support the mobile application while maintaining security, performance, and scalability.

2. API Principles

Every API should follow these principles:

RESTful design.

Consistent response format.

Authentication where required.

Meaningful error messages.

Versioned endpoints (/api/v1/...).

Input validation on every request.

3. Authentication

POST

/api/v1/auth/register

Creates a new account.

POST

/api/v1/auth/login

Authenticates an existing user.

POST

/api/v1/auth/logout

Logs the current user out.

GET

/api/v1/auth/me

Returns the authenticated user's profile.

4. User APIs

GET

Retrieve user profile.

PATCH

Update profile.

Fields may include:

Display name

Profile picture

Notification preferences

5. Course APIs

GET

List all available courses.

GET

Retrieve course information.

POST

Join course.

DELETE

Leave course.

6. Assignment APIs

GET

Retrieve assignment feed.

Supports:

Pagination

Course filter

Search

Due date sorting

GET

Retrieve single assignment.

Returns:

Question

Attachments

Context

Discussion summary

Status

POST

Upload assignment.

Requires:

Course

Title

Question

Deadline

Attachments (optional but at least one content source)

PATCH

Edit assignment.

Only uploader, trusted contributors, or moderators may edit.

DELETE

Delete assignment.

Restricted.

7. Attachment APIs

Upload image.

Upload PDF.

Retrieve attachment.

Delete attachment.

Uploads should be scanned and validated before storage.

8. Context APIs

Create context.

Update context.

Delete context.

Retrieve context.

Multiple context entries are allowed.

9. Discussion APIs

Retrieve discussion.

Create comment.

Reply.

Edit own comment.

Delete own comment.

Future moderation endpoints can extend these.

10. Assignment Status APIs

Retrieve personal status.

Update status.

Retrieve anonymous class statistics.

Example response:

12 Not Started

28 In Progress

14 Finished

9 Submitted

11. Notification APIs

Retrieve notifications.

Mark one as read.

Mark all as read.

Delete notification.

Notifications should support pagination.

12. Contribution APIs

Retrieve contribution history.

Retrieve contribution score.

Retrieve contributor badge.

These are primarily for displaying user achievements.

13. Reporting APIs

Submit report.

Retrieve report status (admin only).

Resolve report (admin).

14. Emergency Answer APIs

Request unlock status.

Check eligibility.

Record usage.

Return appropriate response.

The backend always decides eligibility.

The frontend must never bypass these rules.

15. Search API

Supports searching by:

Course

Assignment title

Keywords

Topic

Future:

Advanced filters.

16. Error Format

Every error response should follow one structure.

Example:

```
{
  "success": false,
  "message": "Assignment not found.",
  "code": "ASSIGNMENT_NOT_FOUND"
}
```

This makes frontend handling much simpler.

17. Success Format

Likewise, successful responses should be consistent.

Example:

```
{
  "success": true,
  "data": { ... }
}
```

18. Rate Limiting

To reduce abuse:

Limit login attempts.

Limit uploads per minute.

Limit comments per minute.

Limit reports per minute.

The exact limits can be tuned later.

19. Security

The backend must:

Validate all input.

Authenticate all protected routes.

Authorize sensitive actions.

Sanitize user-generated content.

Reject invalid file types.

Reject oversized uploads.

Never expose internal database IDs where UUIDs are used.

20. Logging

Every important action should be logged.

Examples:

Login

Logout

Assignment created

Assignment edited

Assignment deleted

Emergency Answer used

User reported

These logs assist debugging and moderation.

21. Performance

Every list endpoint should support:

Pagination.

Sorting.

Filtering.

Future caching.

Avoid returning unnecessary data.

22. Versioning

All APIs should include versioning from the beginning.

Example:

/api/v1/...

This allows future improvements without breaking existing clients.

23. Future APIs

Reserved for later:

Timetable integration.

Calendar sync.

Institution integration.

Reference assignment archive.

Study groups.

Analytics.

End of Document 6

One thing I noticed while writing this

We should probably introduce one more concept in a future document:

Assignment Verification

Right now, anyone can upload an assignment.

That's great for keeping the app community-driven, but what happens if two students upload slightly different versions?

Imagine:

Student A uploads: Question 5 = Solve $x^2 = 16$

Student B uploads: Question 5 = Solve $x^2 = 64$

Which one is correct?

I think Version 1 should include a lightweight verification system.

For example:

The original uploader is marked.

Other students can confirm, "This matches what the lecturer gave."

Trusted Contributors can verify it.

If multiple trusted contributors agree, the assignment gets a small Verified badge.

That way, you're not relying on popularity—you're building confidence in the accuracy of the information. I think that fits the philosophy we've developed throughout the project.